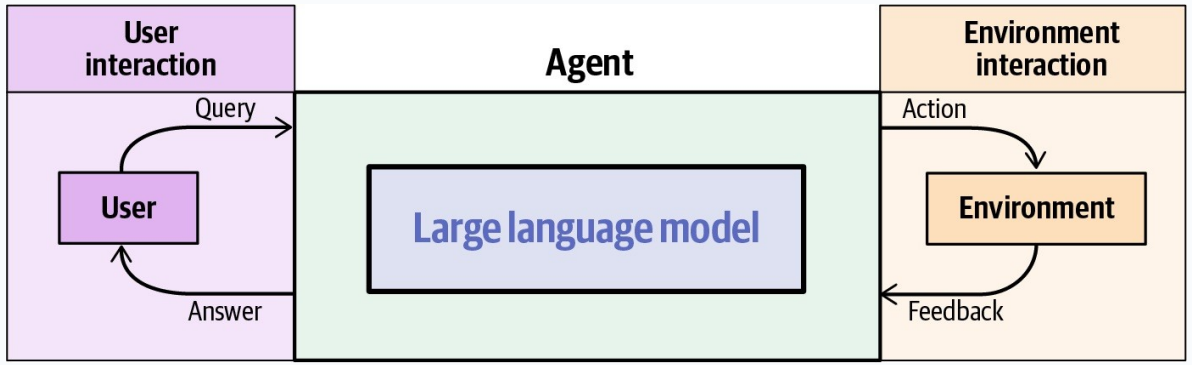
Large Language Models: The Engine of AI Agents

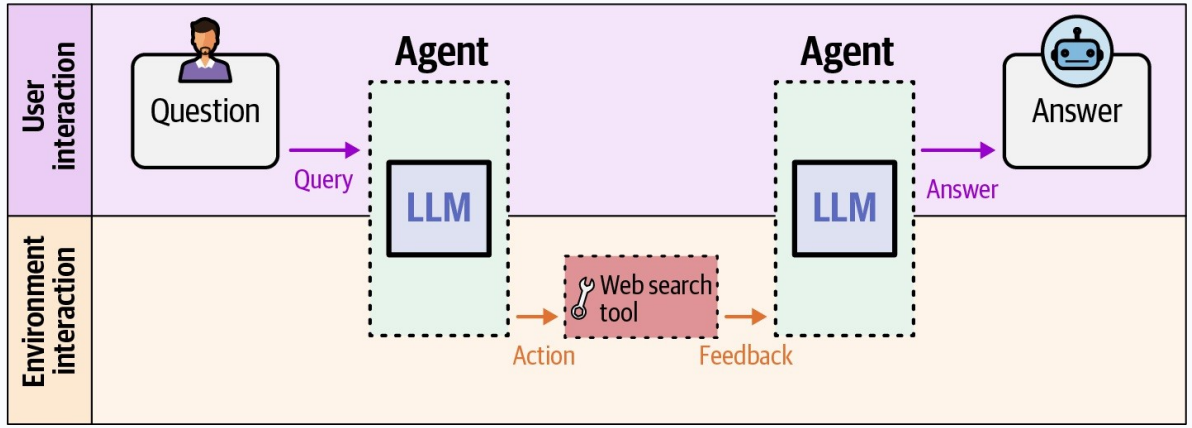
- A language model interprets observations and produces the next response or action proposal.
- In an agent, the model operates within a software system that can execute tool calls and feed the results back as observations.
- This distinction separates generating a plausible instruction from taking a verified action.



Original textbook illustration · PDF p. 50

From a User Question to an External Action

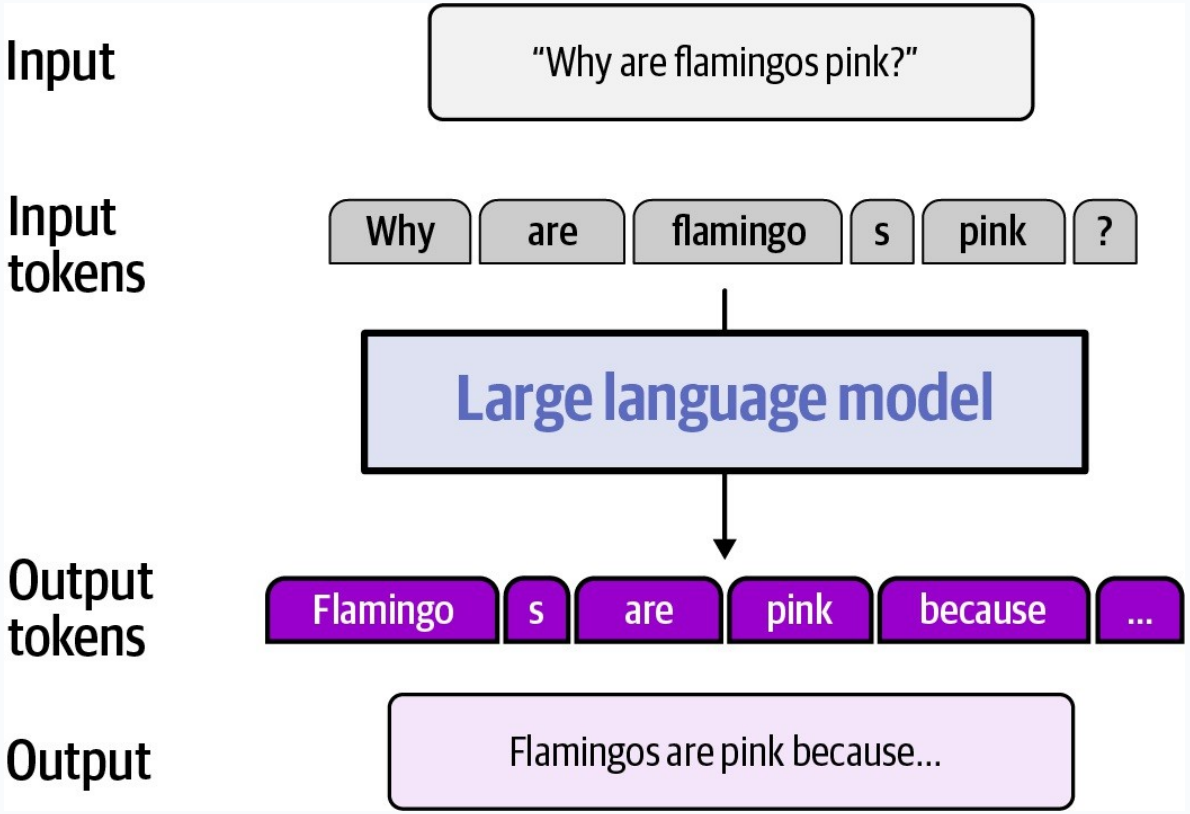
- An agent can decide that the answer requires information not contained in its immediate context.
- It sends a structured request to a search tool, receives retrieved material, and uses the result to compose its answer.
- Tool execution occurs outside the language model; the model selects and interprets the operation.



Original textbook illustration · PDF p. 51

Language Models Process Tokens

- Text is divided into tokens before being processed by a language model. A token may represent a word, a word fragment, punctuation, or another unit.
- Tokenization influences context usage and generation cost. A familiar word can be split into multiple tokens.
- Both the prompt and the generated response are represented as sequences of tokens.



Original textbook illustration · PDF p. 53

Autoregressive Text Generation

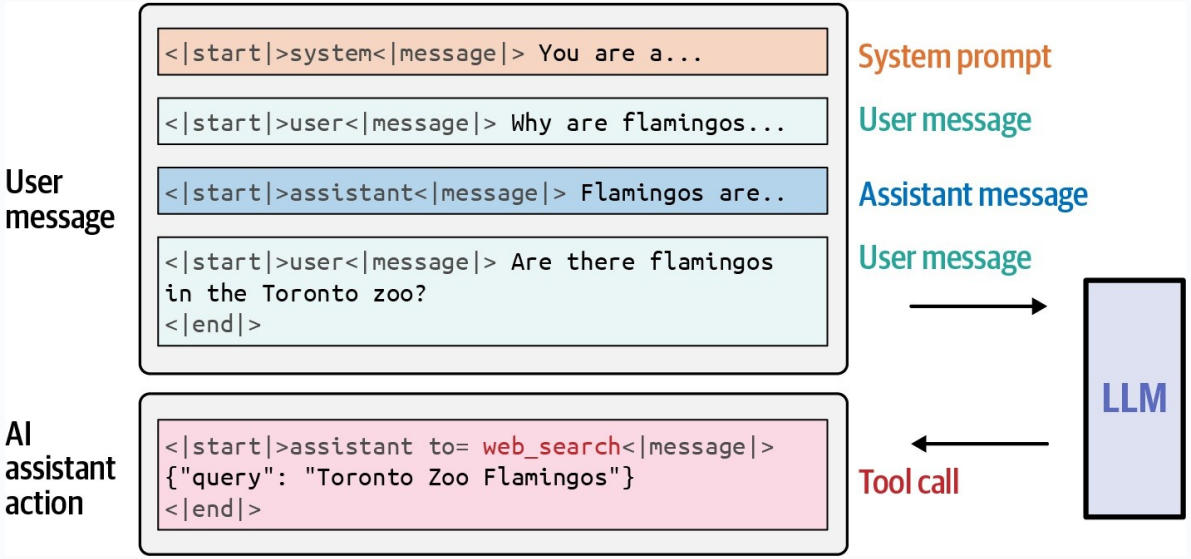
- A language model predicts a probability distribution for the next token given all tokens currently available.
- After a token is selected, it is appended to the sequence and the prediction process repeats.
- This iterative mechanism can produce prose, program code, or structured agent messages.



Original textbook illustration · PDF p. 54

Messages, Roles, and Conversation Structure

- A chat-oriented model receives structured messages rather than an undifferentiated block of text.
- System instructions, user messages, assistant replies, and tool results may have different roles and boundaries.
- Correct message formatting helps the model distinguish instructions, evidence, and outputs.



Original textbook illustration · PDF p. 56

A Minimal Language-Model Interface

- An agent needs a dependable interface for submitting messages to a model and receiving a response.
- A model wrapper can centralize model selection, temperature, response parsing, and other invocation details.
- Separating the interface from the agent makes the surrounding architecture easier to change.

Model Responses and Agent Trajectories

- A model response may contain user-facing content, reasoning-related fields, or a structured action request.
- A step records what the agent decided and what it observed; a trajectory is a sequence of such steps.
- This structure supports debugging, traceability, and later extensions such as tool use.

The Foundation: Pre-Training

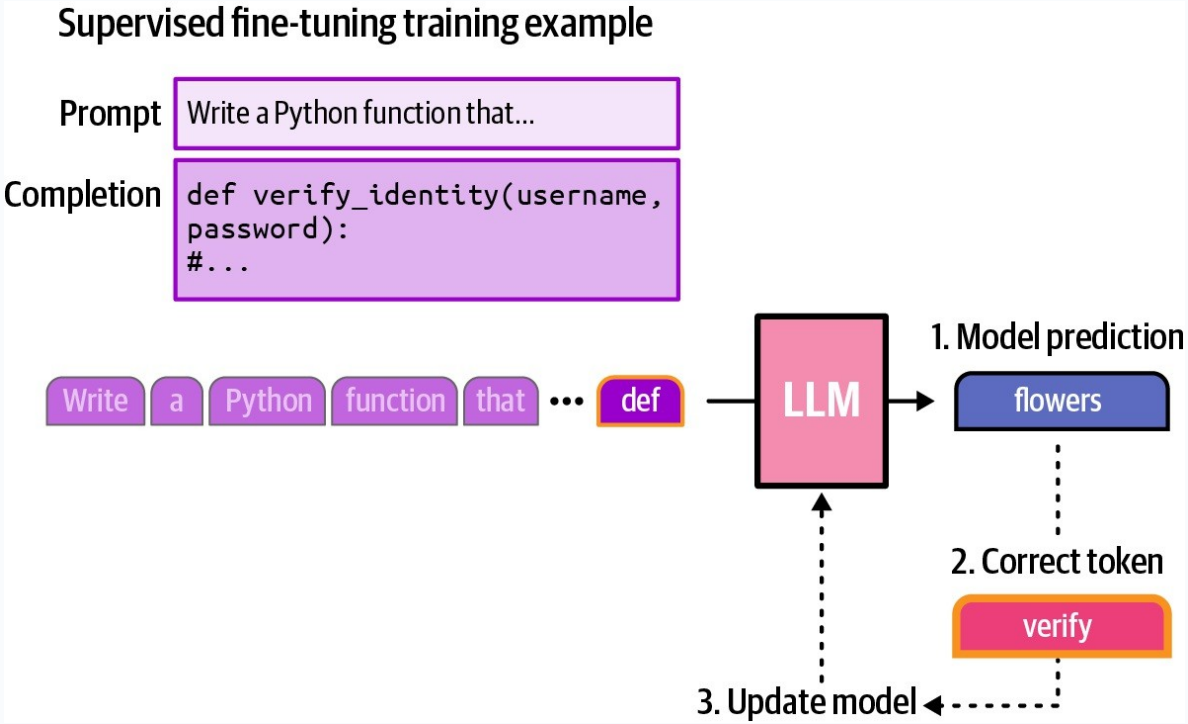
- Pre-training exposes a language model to large curated mixtures of text, code, and other supported data.
- The objective is to learn statistical structure and patterns that support prediction across diverse contexts.
- Data composition and filtering influence what the resulting model can represent and how reliably it responds.

Learning Through Next-Token Prediction

- During pre-training, the model predicts the next token in a training sequence.
- The prediction is compared with the target token; the error drives updates to the model parameters.
- Repeated optimization improves the model’s ability to assign useful probabilities to token sequences.

Supervised Fine-Tuning (SFT)

- Supervised fine-tuning uses examples of prompts paired with desired completions.
- The model learns to produce responses that better follow instructions and match useful assistant behavior.
- SFT changes the model from a general sequence predictor into a more effective instruction follower.



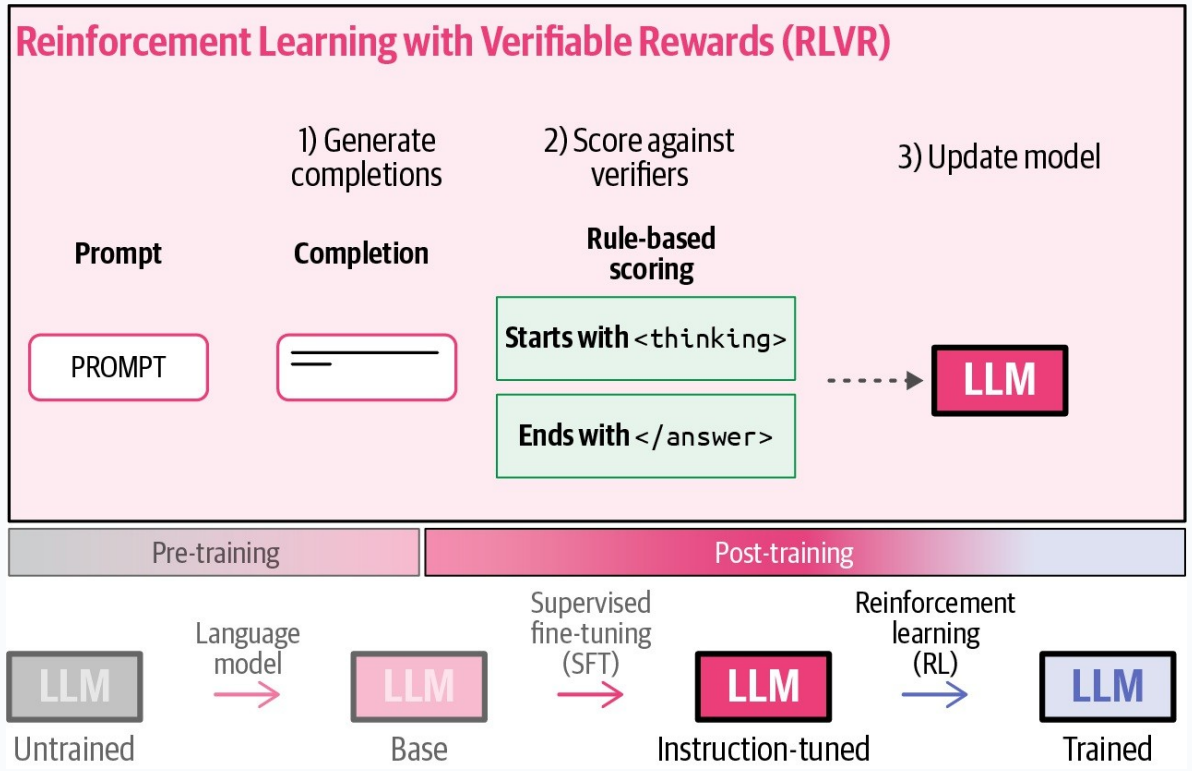
Original textbook illustration · PDF p. 69

Learning From Preferences

- Preference-based training compares alternative model outputs and uses judgments about which response is better.
- A training signal rewards responses aligned with the preferred behavior and discourages weaker alternatives.
- This stage can improve response usefulness, format, and instruction adherence beyond SFT.

Reinforcement Learning With Verifiable Rewards

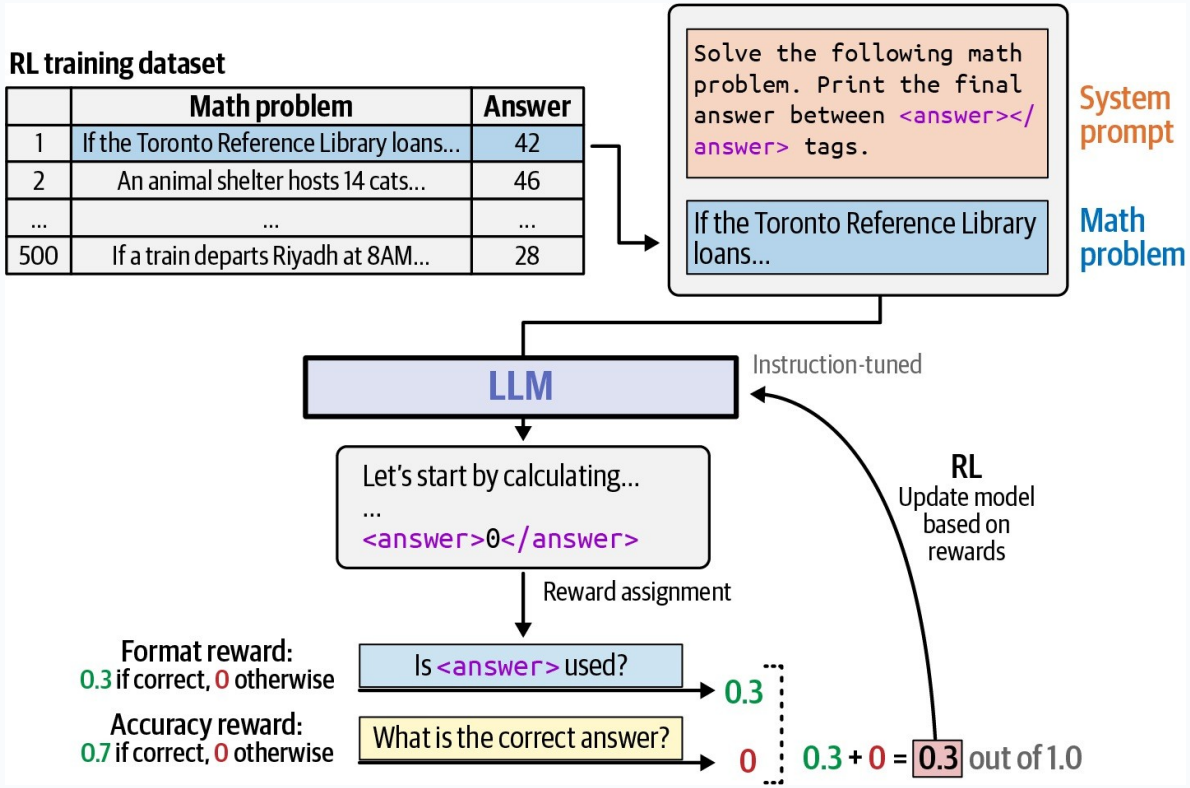
- Some tasks allow the correctness of an output to be checked automatically, such as mathematics or code tests.
- A verifier supplies a reward based on measurable success instead of relying only on subjective preference.
- Verifiable rewards make it possible to optimize performance on tasks with clear success criteria.



Original textbook illustration · PDF p. 72

A Worked Example of Verifiable Reward

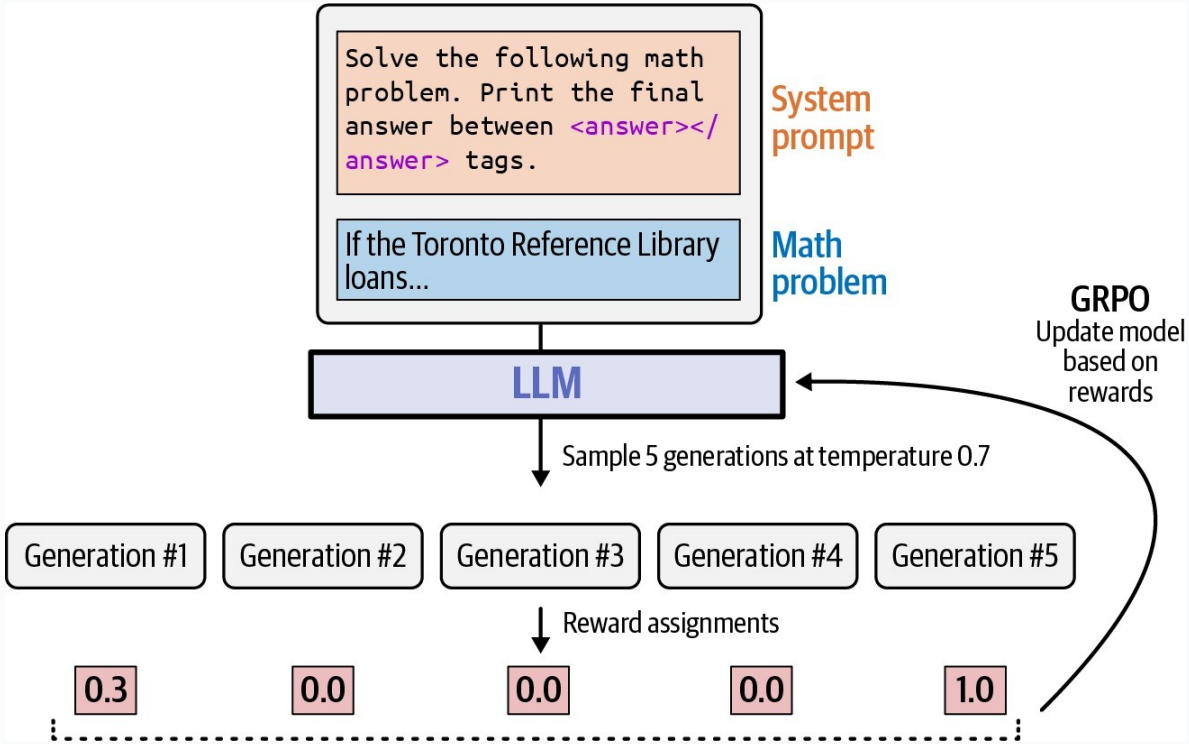
- The model attempts a problem and produces an answer in a required format.
- A verifier checks properties such as formatting and correctness, then assigns reward values.
- Training increases the likelihood of behavior that consistently satisfies the verification criteria.



Original textbook illustration · PDF p. 74

Group Relative Policy Optimization

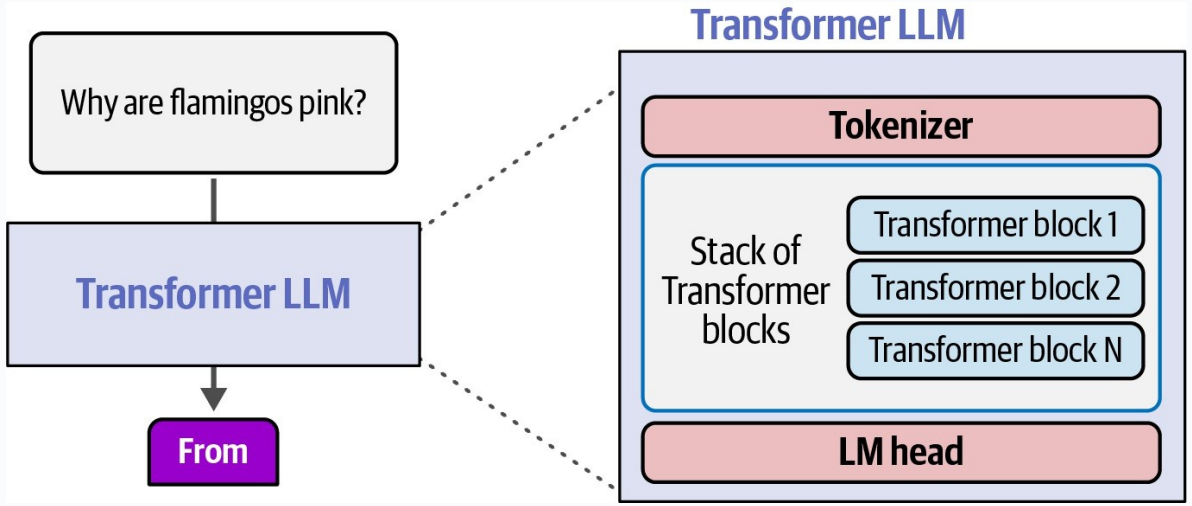
- A training procedure can sample several candidate responses to the same prompt.
- Each candidate receives a reward, and its quality is evaluated relative to other candidates in the group.
- The model is updated to favor stronger responses without requiring a separate value estimate for every step.



Original textbook illustration · PDF p. 76

Inside a Modern LLM

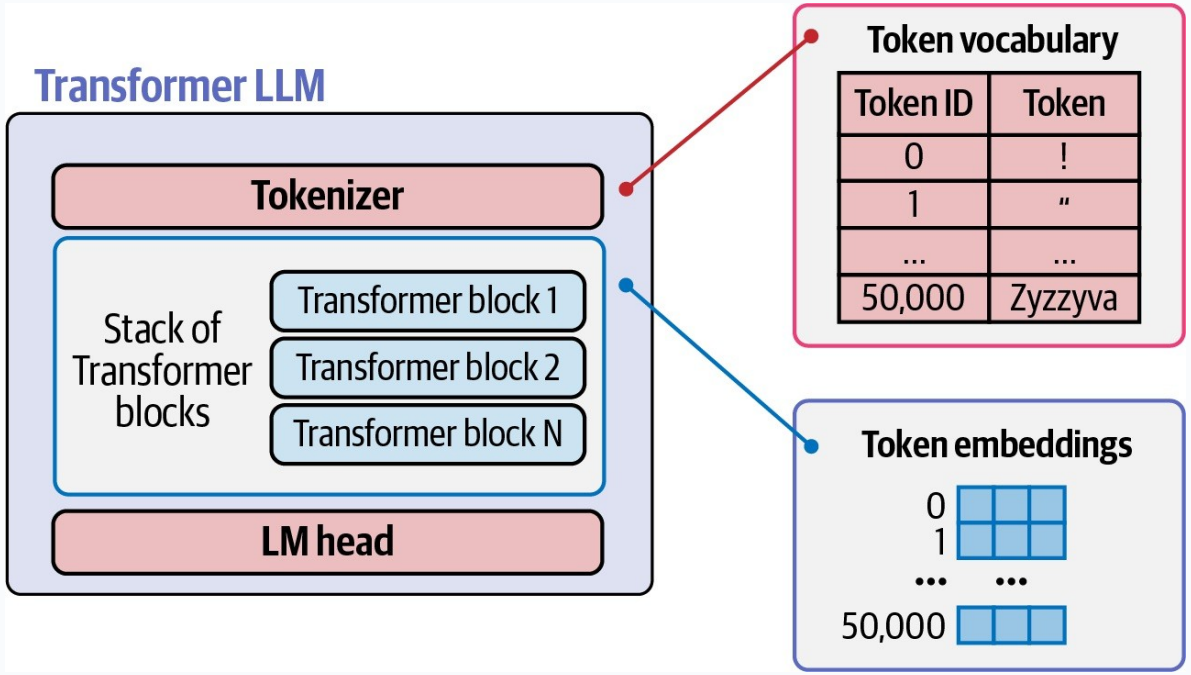
- A typical language-model pipeline includes tokenization, token embeddings, a stack of Transformer blocks, and an output projection.
- Embeddings convert discrete tokens into numerical vectors that can be processed by neural-network layers.
- The final representation is transformed into scores over the vocabulary to support next-token prediction.



Original textbook illustration · PDF p. 77

Token Embeddings: Text Becomes Vectors

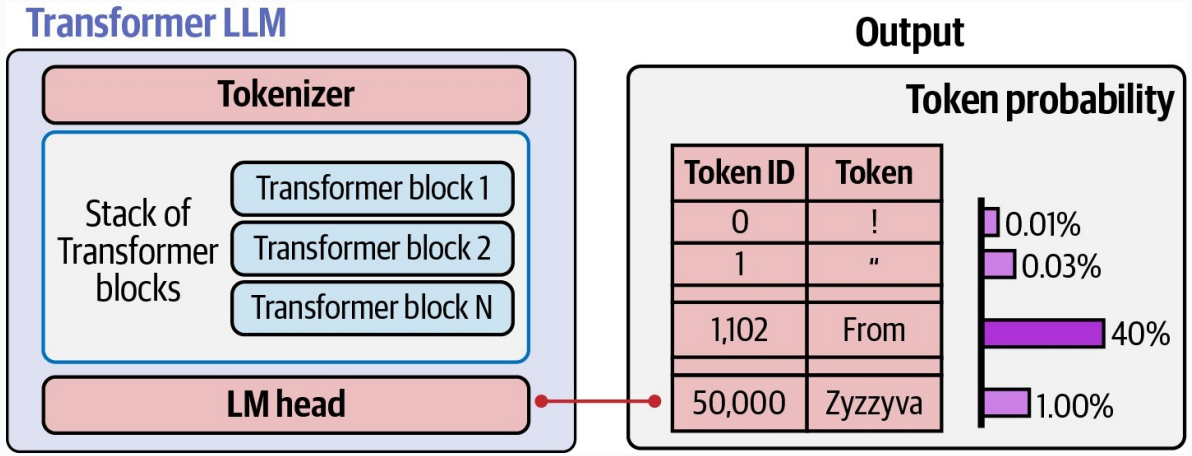
- Each token identifier is mapped to a learned vector of numerical values.
- These vectors form the input representation that the Transformer processes and updates.
- Representations are contextualized as information flows through the model's layers.



Original textbook illustration · PDF p. 78

The Language-Model Head

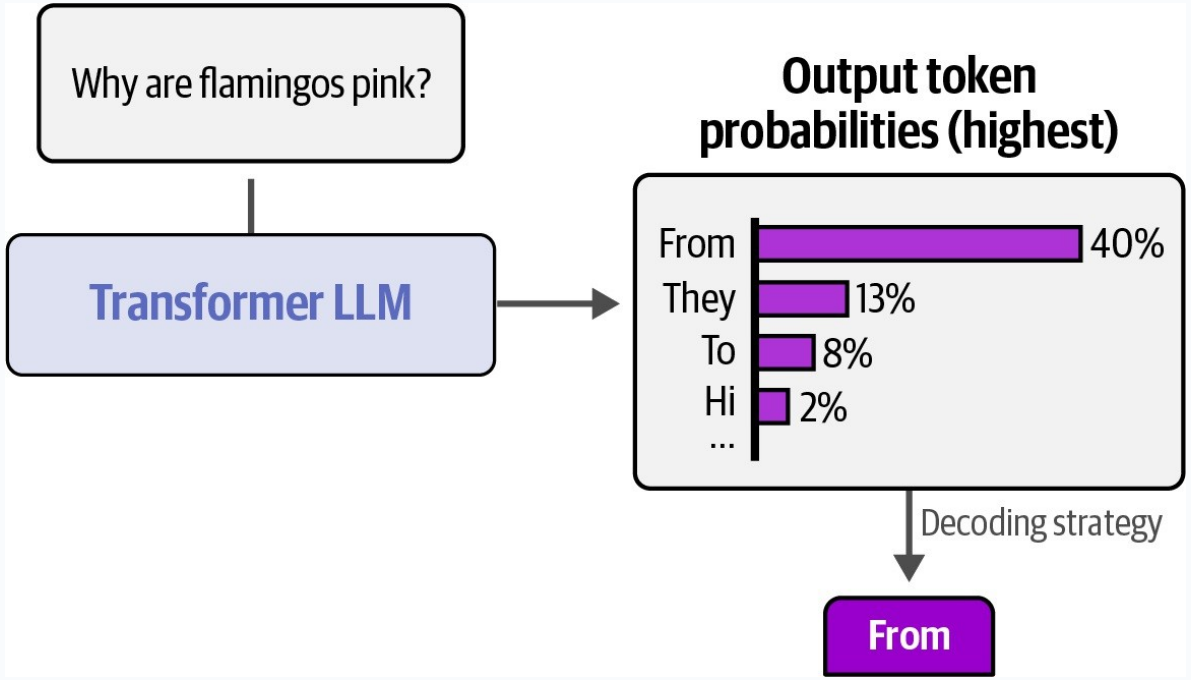
- The final hidden representation is projected into a score for each token in the vocabulary.
- After normalization, the scores define a probability distribution over possible next tokens.
- The output head is the bridge between internal numerical representations and generated language.



Original textbook illustration · PDF p. 79

Decoding and Temperature

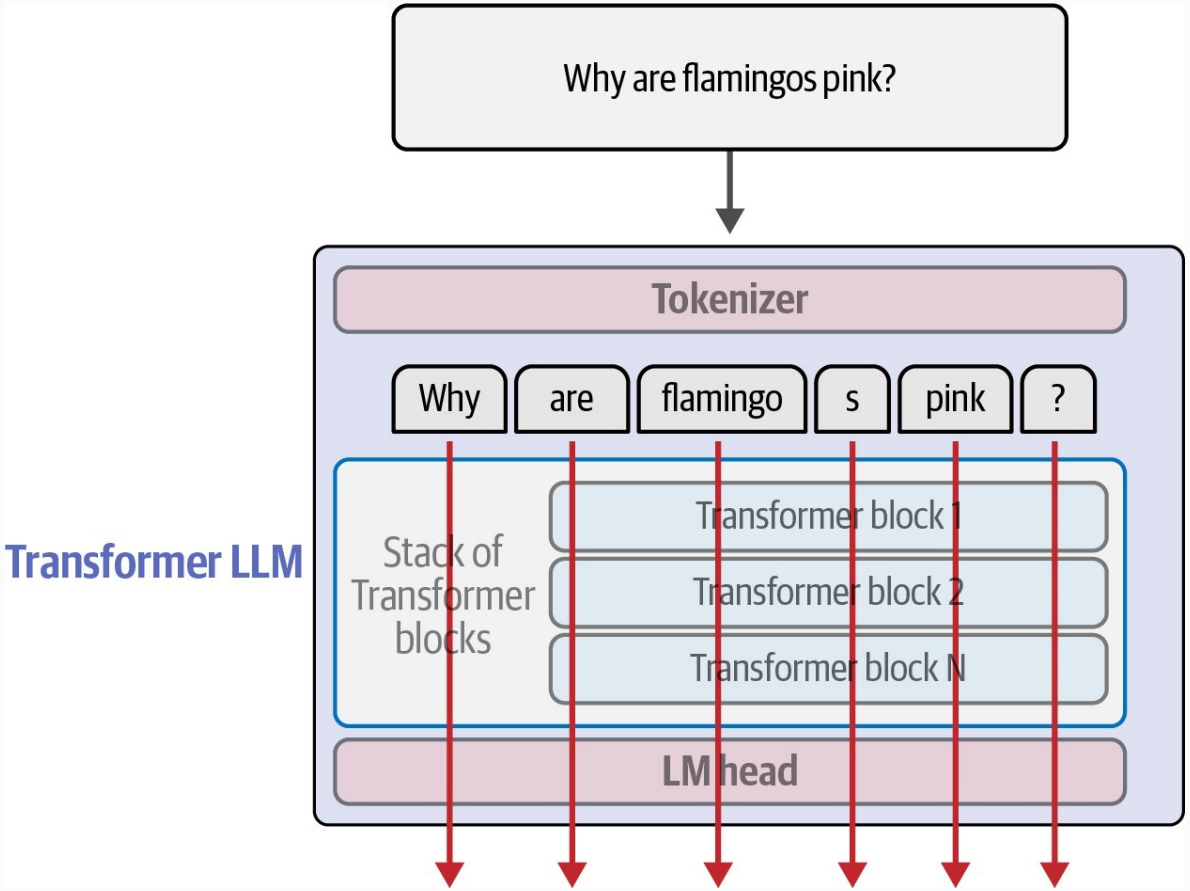
- Decoding determines how a next token is selected from the model's output distribution.
- Lower temperatures generally concentrate selection on high-probability tokens; higher temperatures produce more varied samples.
- The best decoding setting depends on the task: precision and repeatability versus diversity.



Original textbook illustration · PDF p. 80

Parallel Processing During Prompt Ingestion

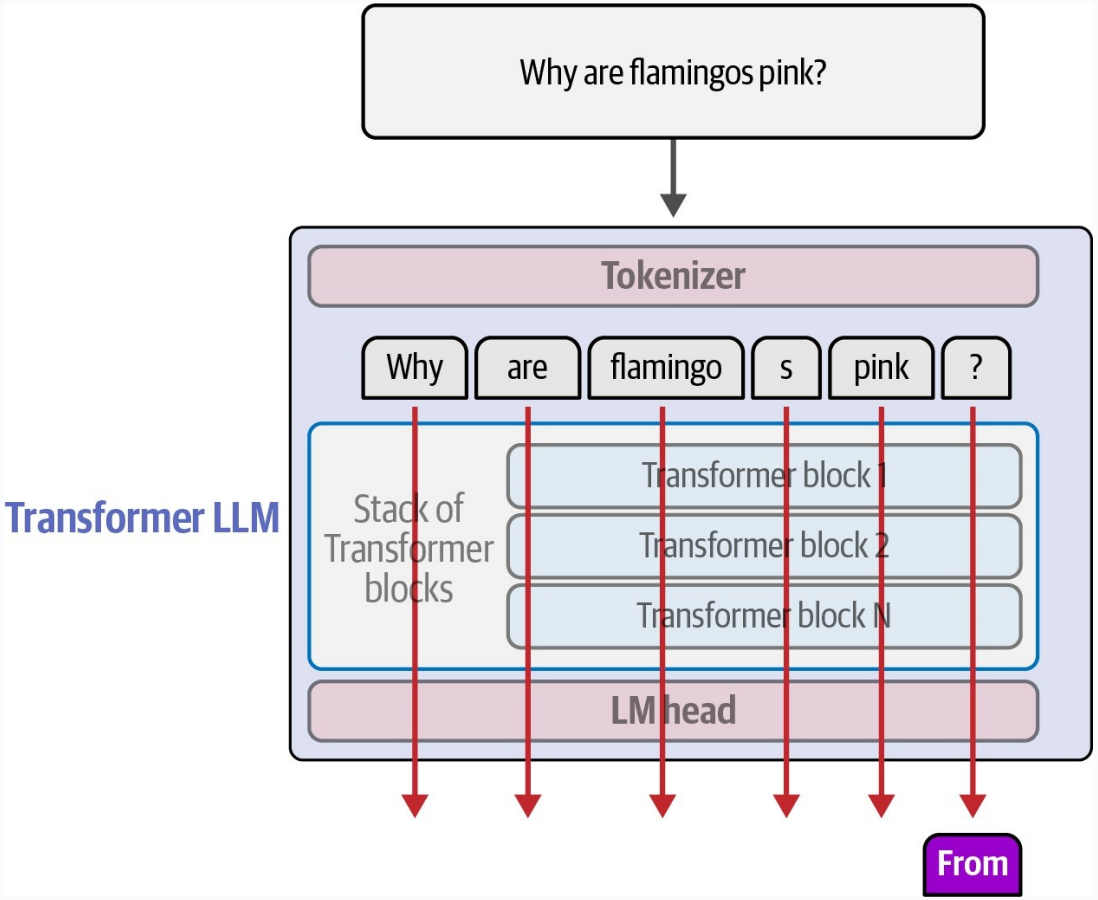
- Transformer layers process representations for many prompt positions in parallel.
- Causal constraints prevent a position from attending to future positions in the sequence.
- Prompt processing differs from generation, which adds tokens sequentially.



Original textbook illustration · PDF p. 81

Why the Last Token Matters During Generation

- When generating a new token, the model uses the final position's representation to compute the next-token distribution.
- Earlier positions still matter because their information influences the final position through attention.
- The same Transformer stack is reused as the sequence grows.



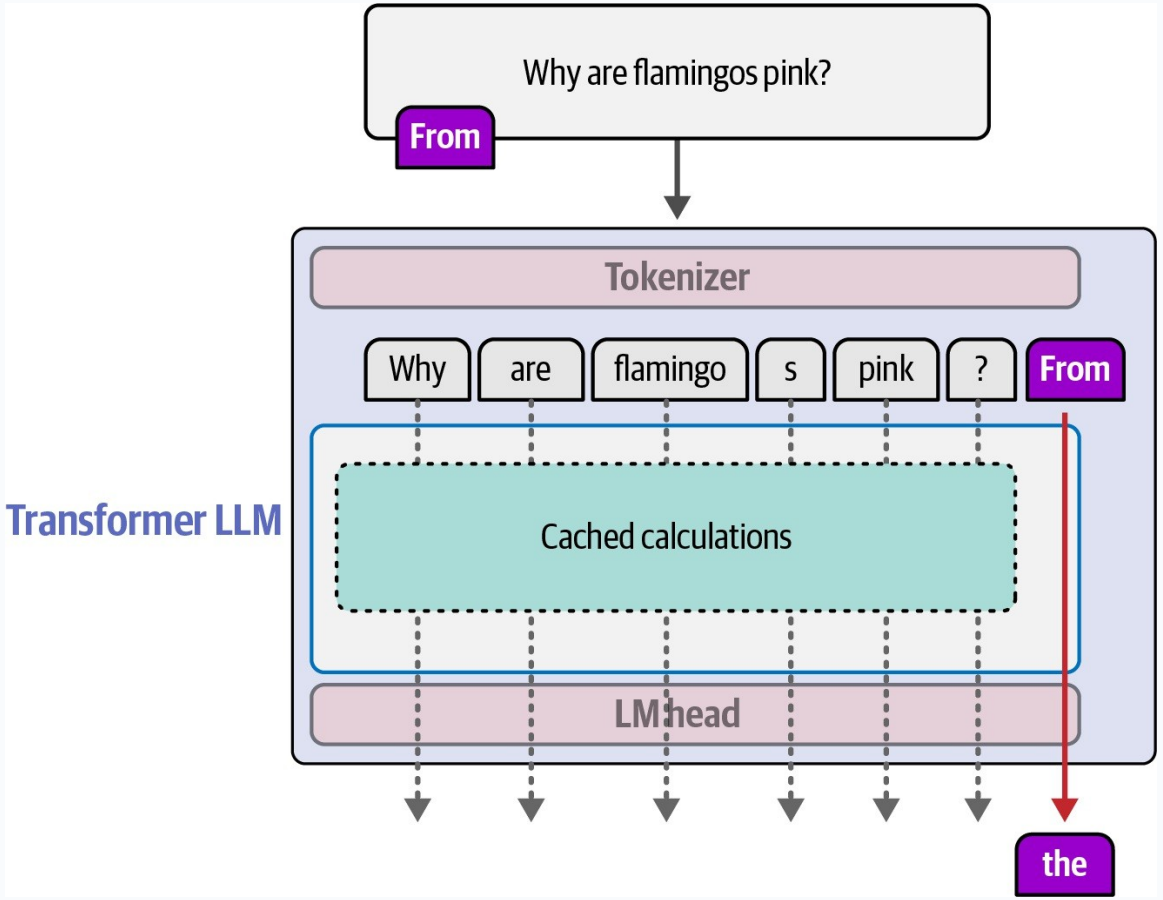
Original textbook illustration · PDF p. 83

Context Window and Agent Design

- The context window limits how much tokenized material a model can consider in one invocation.
- Instructions, conversation history, retrieved evidence, and tool outputs all consume this capacity.
- Agents must manage context carefully, preserving relevant information without overwhelming the model.

KV Cache: Reusing Previous Computation

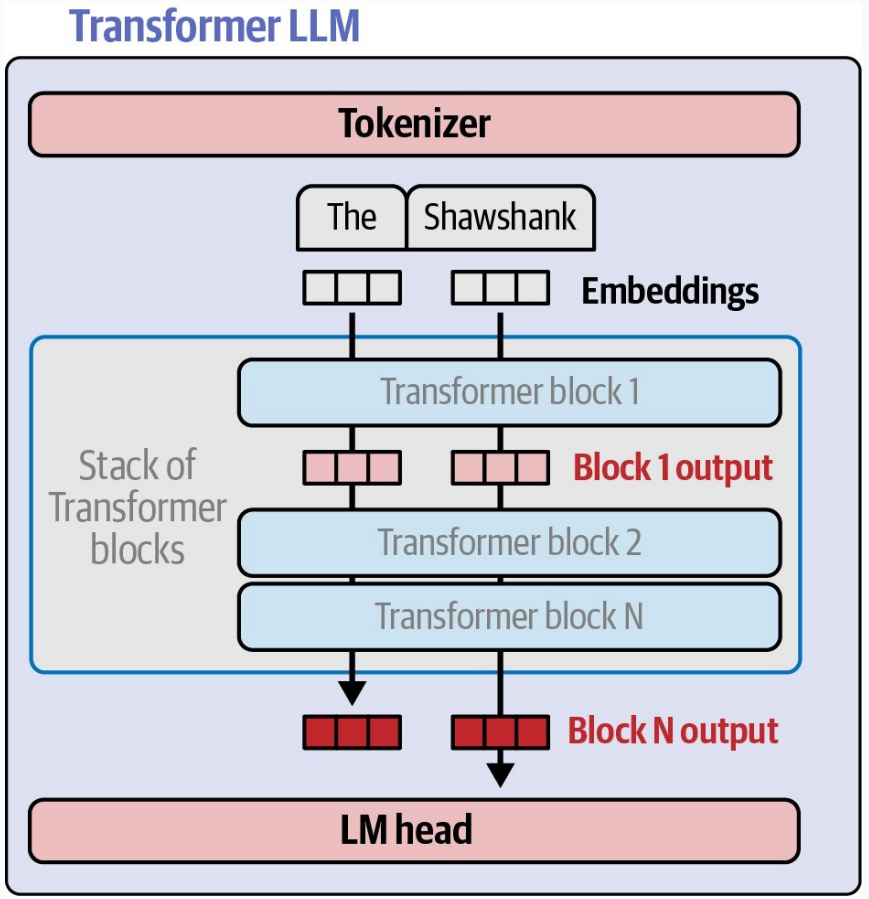
- Autoregressive generation would be wasteful if every previously processed token were fully recomputed at each step.
- A key–value cache retains attention-related representations from earlier positions.
- New generation steps can reuse cached values, improving inference speed at the cost of memory.



Original textbook illustration · PDF p. 85

The Transformer Stack

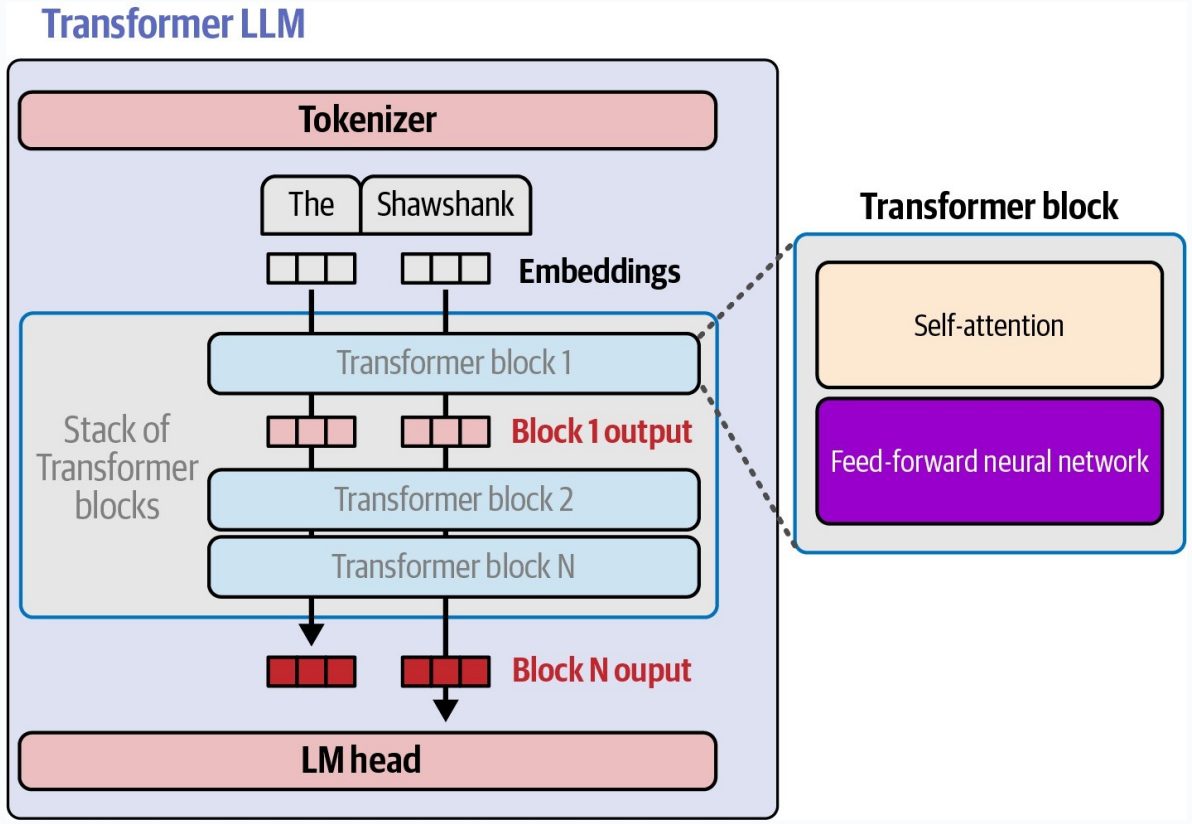
- A token representation passes through successive Transformer blocks.
- Each block modifies the representation by incorporating contextual information and applying learned transformations.
- The final representation encodes information needed to predict the next token.



Original textbook illustration · PDF p. 87

Two Core Components of a Transformer Block

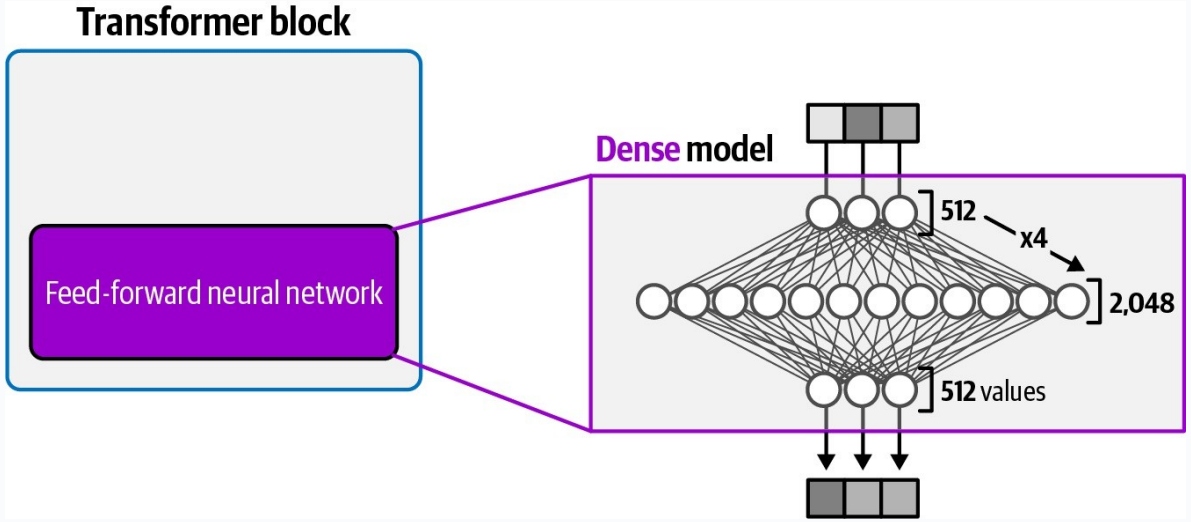
- Self-attention lets a token gather information from relevant positions in the context.
- A feed-forward network transforms the resulting representation at each token position.
- These mechanisms are repeated across many blocks to build progressively useful representations.



Original textbook illustration · PDF p. 89

Feed-Forward Neural Networks

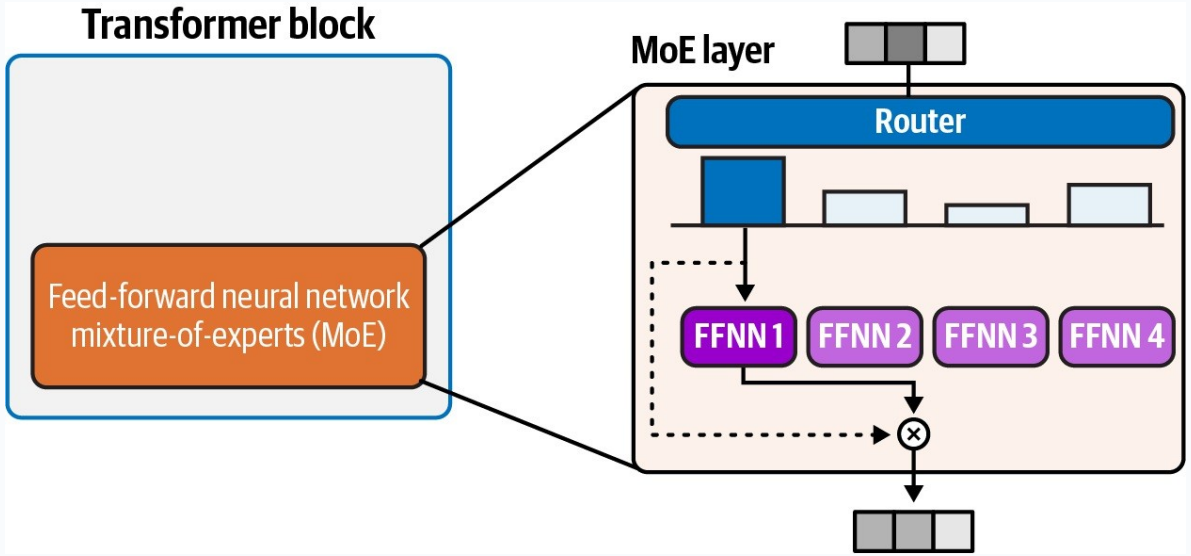
- The feed-forward sublayer applies learned transformations independently to each token position.
- Its intermediate dimensions can be substantially larger than the model's hidden dimension.
- This component contributes significant computation and parameter capacity.



Original textbook illustration · PDF p. 91

From Dense Layers to Mixture of Experts

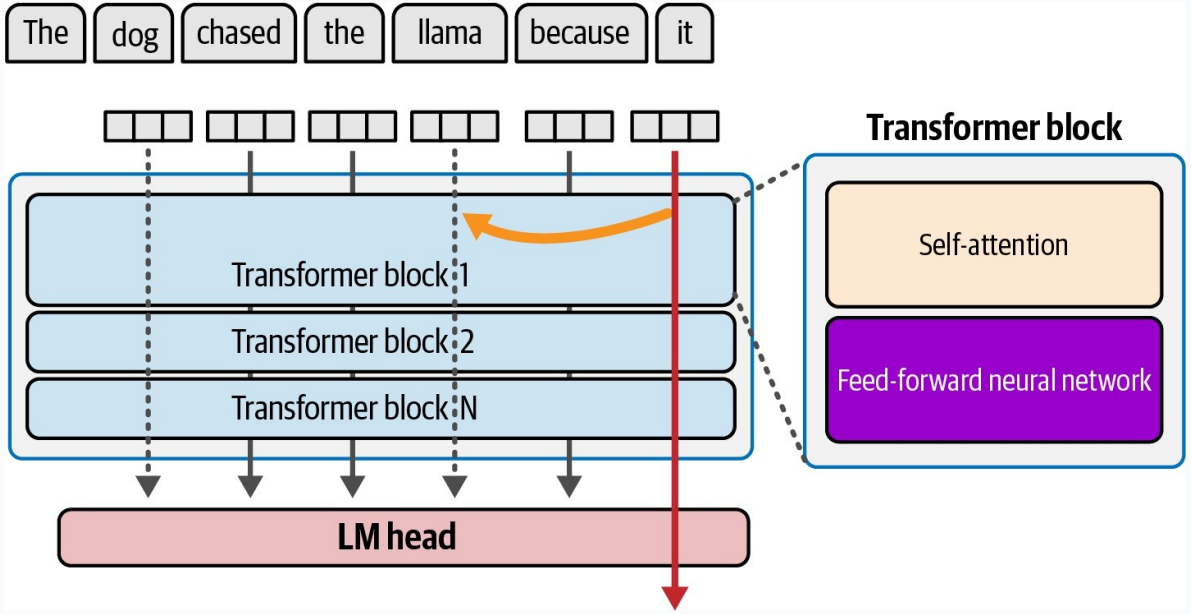
- A dense feed-forward layer activates the same parameter set for each token.
- A mixture-of-experts layer contains multiple expert networks and activates only a selected subset for a token.
- This increases total model capacity without activating every expert for every token.



Original textbook illustration · PDF p. 92

Self-Attention: Context Changes Meaning

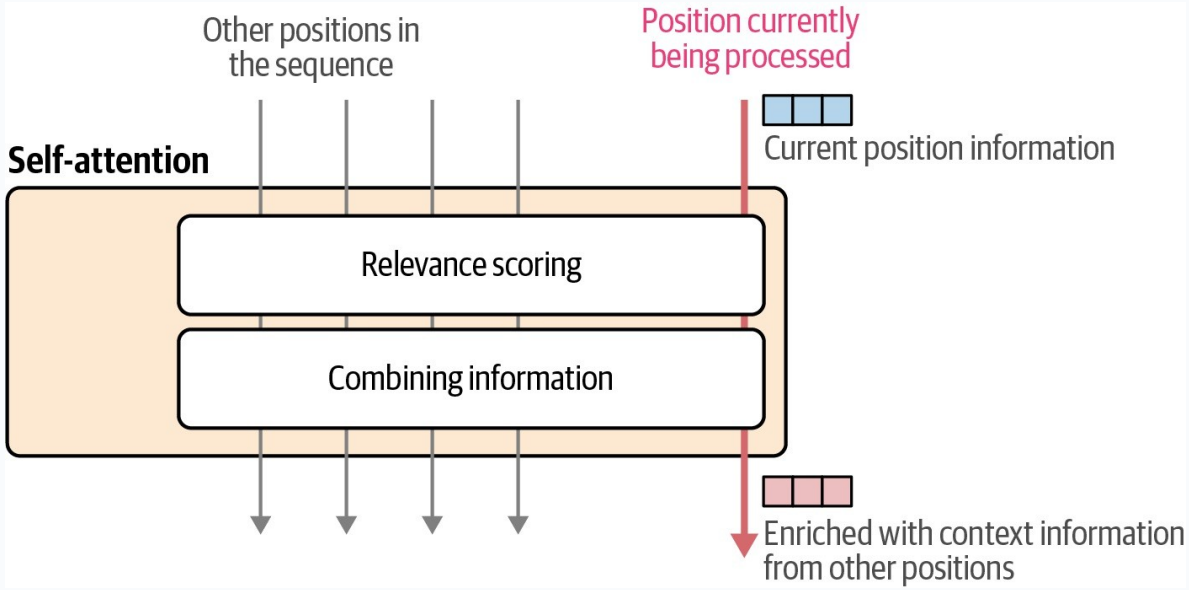
- The interpretation of a token depends on surrounding tokens, not only on its isolated identity.
- Self-attention assigns different relevance weights to context positions and combines their information.
- The result is a context-sensitive representation suited to the current prediction task.



Original textbook illustration · PDF p. 93

Attention in Two Conceptual Steps

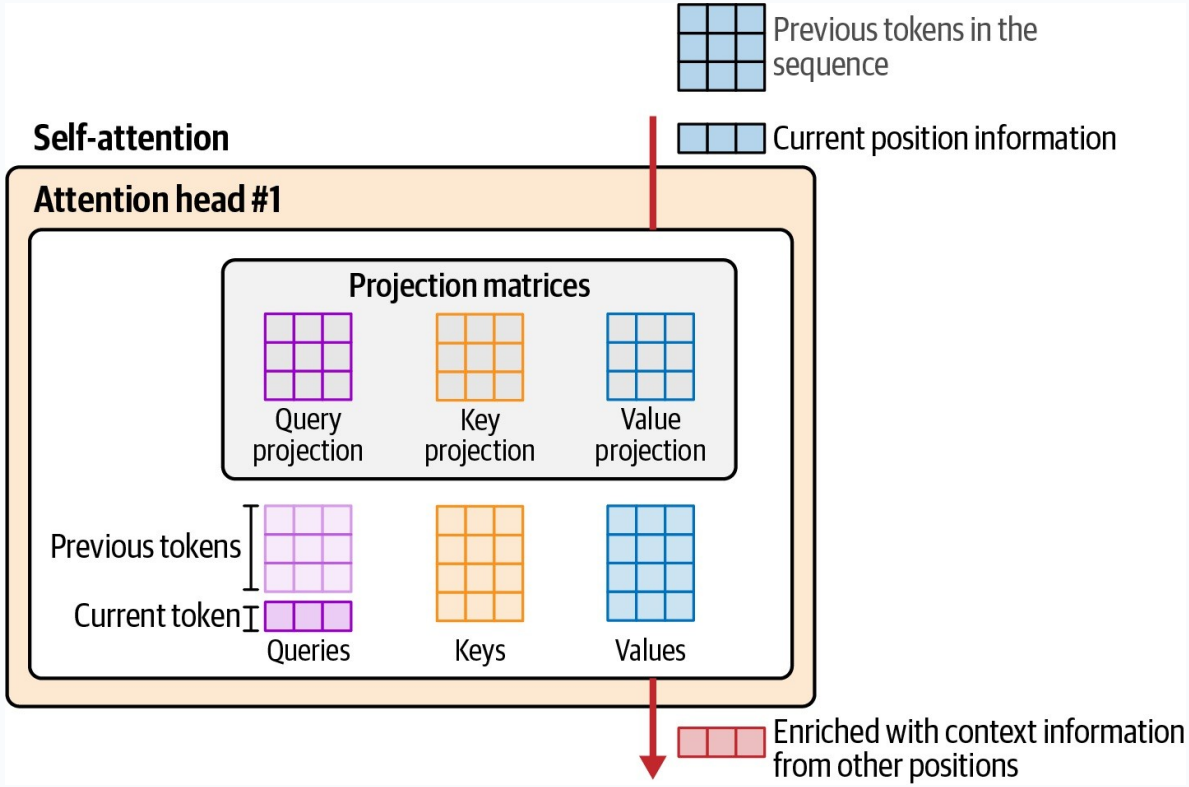
- First, relevance scoring determines how strongly a token should attend to other allowed positions.
- Second, information combining aggregates the relevant value representations according to those scores.
- Together, these operations let the model use selected contextual evidence.



Original textbook illustration · PDF p. 95

Queries, Keys, and Values

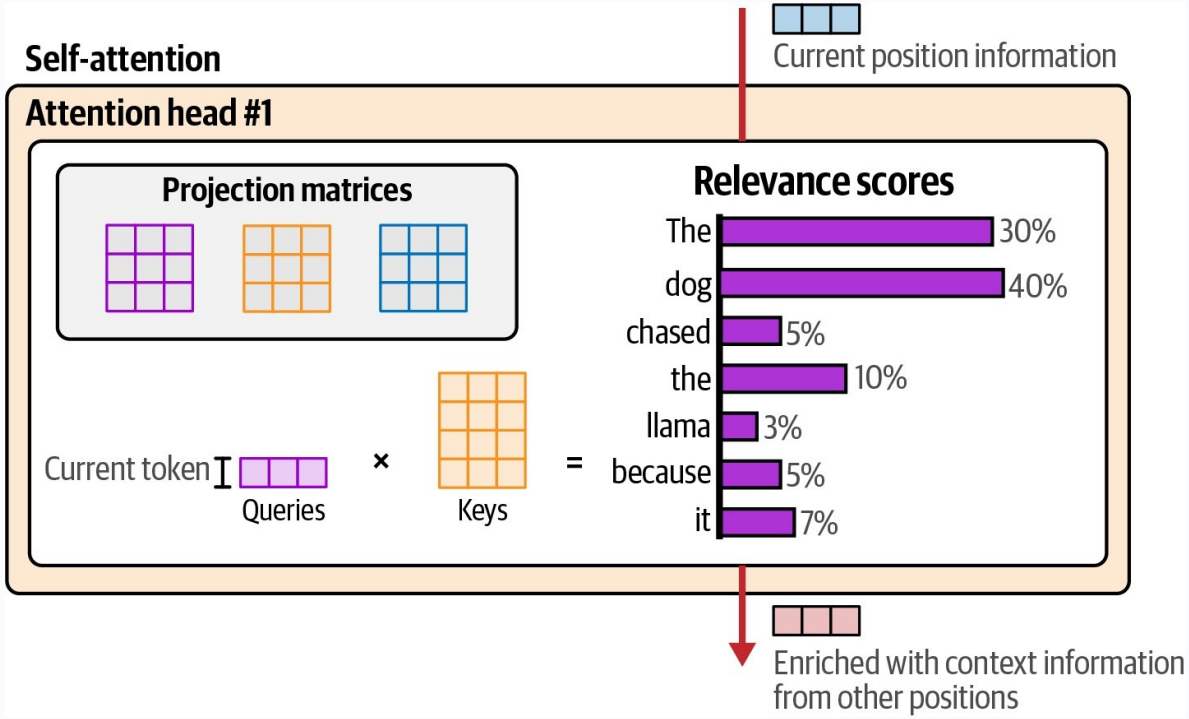
- An attention head transforms token representations into query, key, and value vectors.
- Queries describe what information is sought; keys support relevance matching; values carry the information to combine.
- The vectors are learned projections rather than hand-coded linguistic rules.



Original textbook illustration · PDF p. 97

Computing Attention Scores

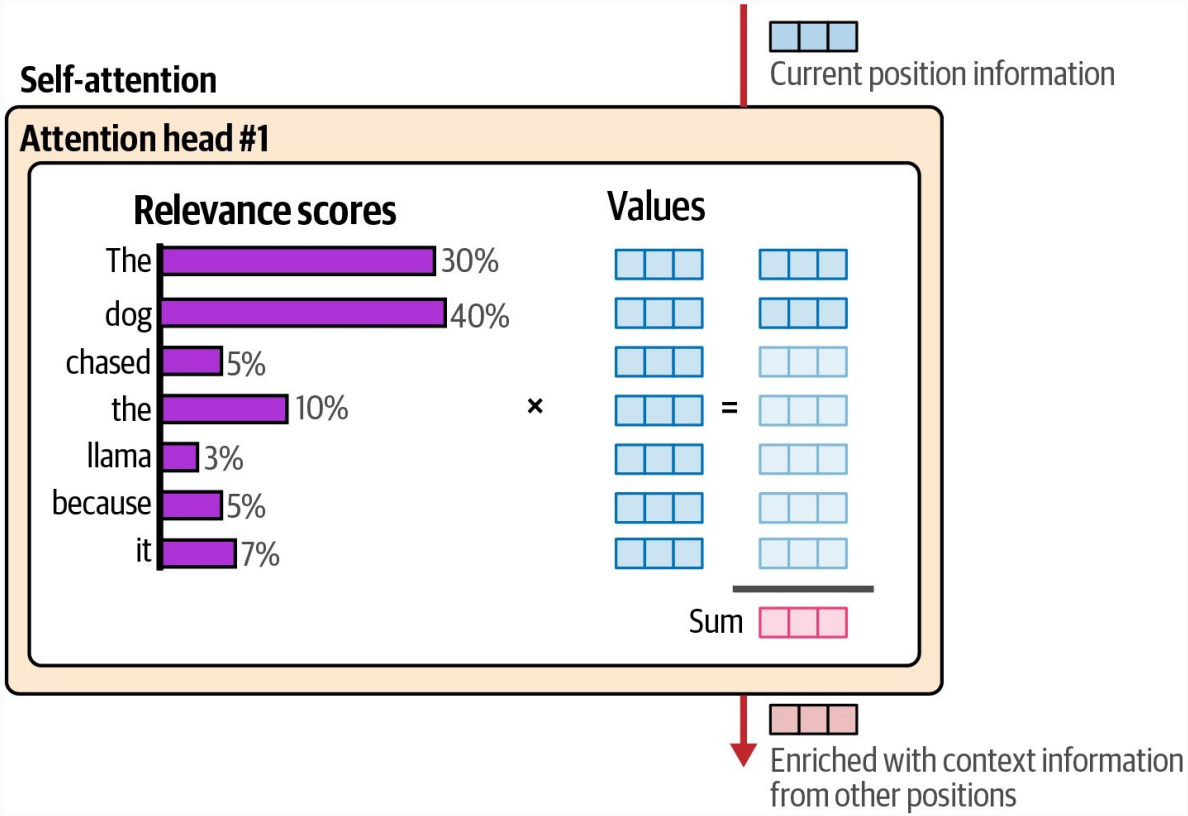
- A query is compared with keys from the allowed context positions.
- The scores are scaled and normalized into attention weights that express relative relevance.
- Causal masking prevents access to positions that occur later in the sequence.



Original textbook illustration · PDF p. 98

Combining Values Into an Attention Output

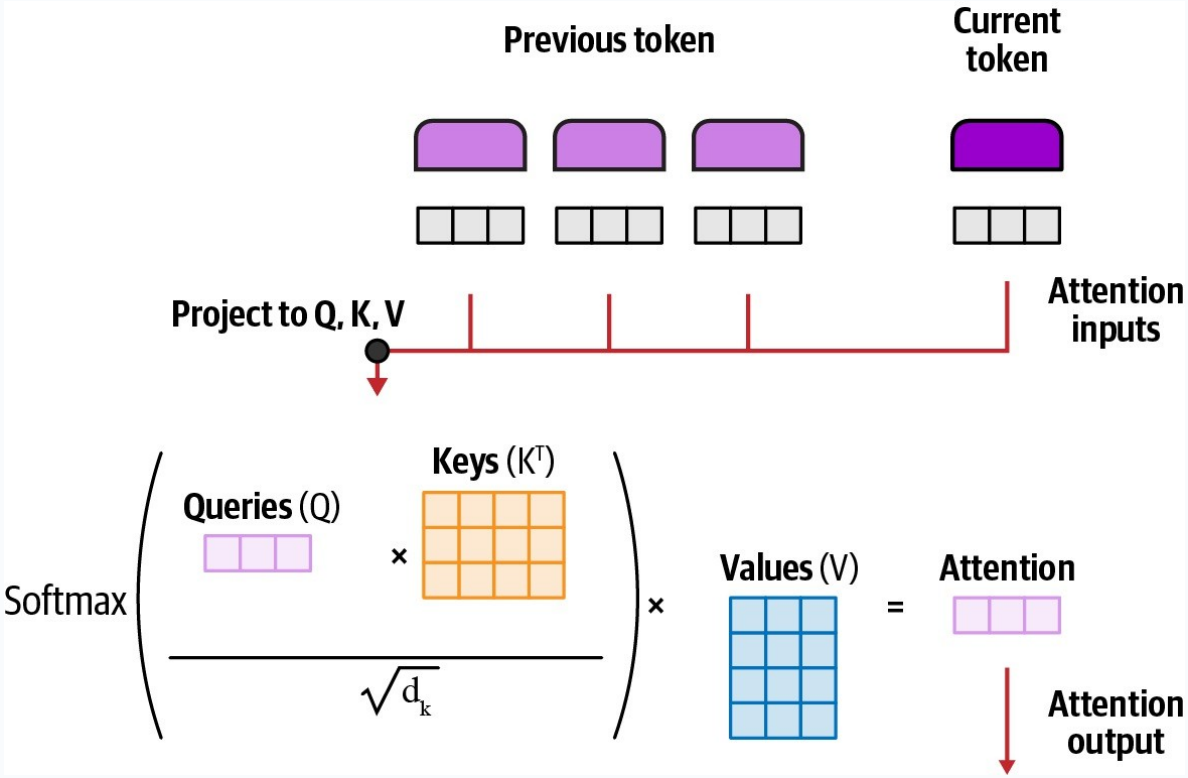
- Each value vector is weighted by the corresponding attention score.
- The weighted vectors are added to produce a new representation for the current position.
- Multiple attention heads can learn different ways to relate positions and combine information.



Original textbook illustration · PDF p. 99

The Attention Computation

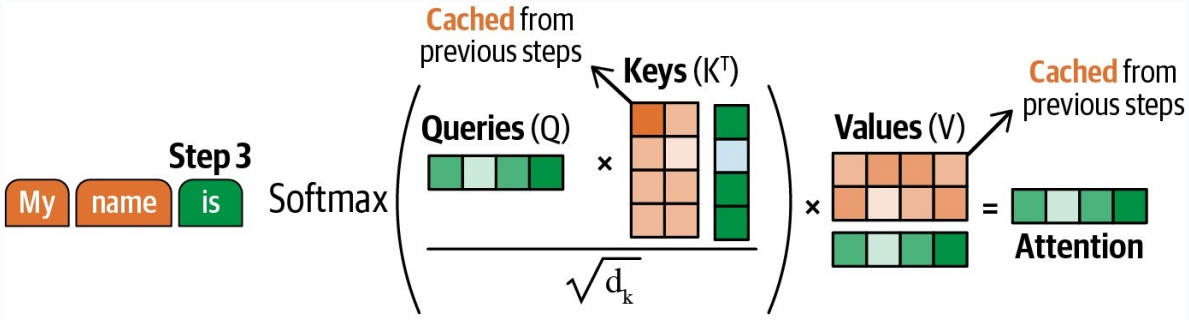
- Scaled dot-product attention can be written as $\text{softmax}(QK^T / \sqrt{d_k})V$.
- The QK^T term measures similarity; scaling controls magnitude; softmax normalizes weights; multiplication by V combines information.
- This compact expression describes the central information-routing operation in a Transformer.



Original textbook illustration · PDF p. 100

Caching Keys and Values

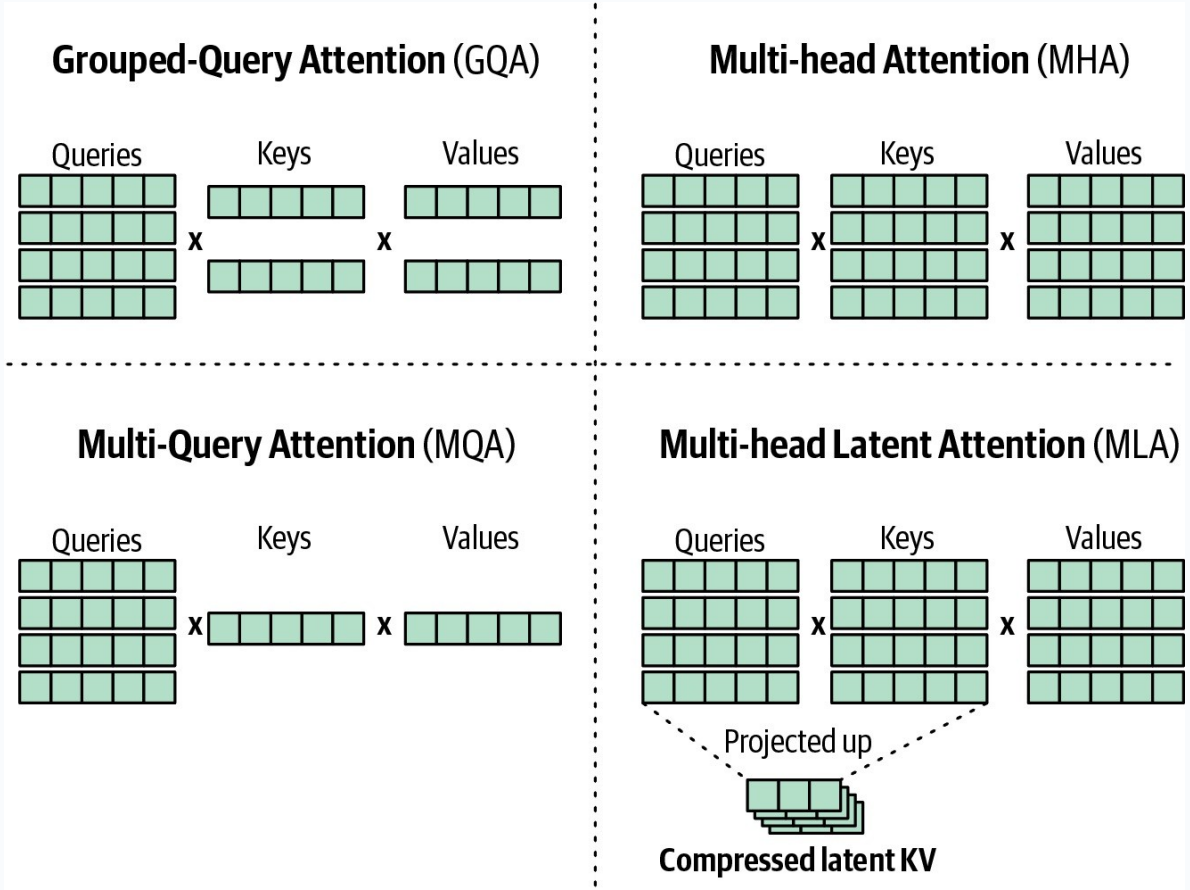
- In incremental generation, earlier keys and values remain useful for future tokens.
- The KV cache stores those vectors so the model only needs to compute the new token's projections.
- Cache size grows with context length and can become a major inference-memory constraint.



Original textbook illustration · PDF p. 102

Attention Variants and Memory Efficiency

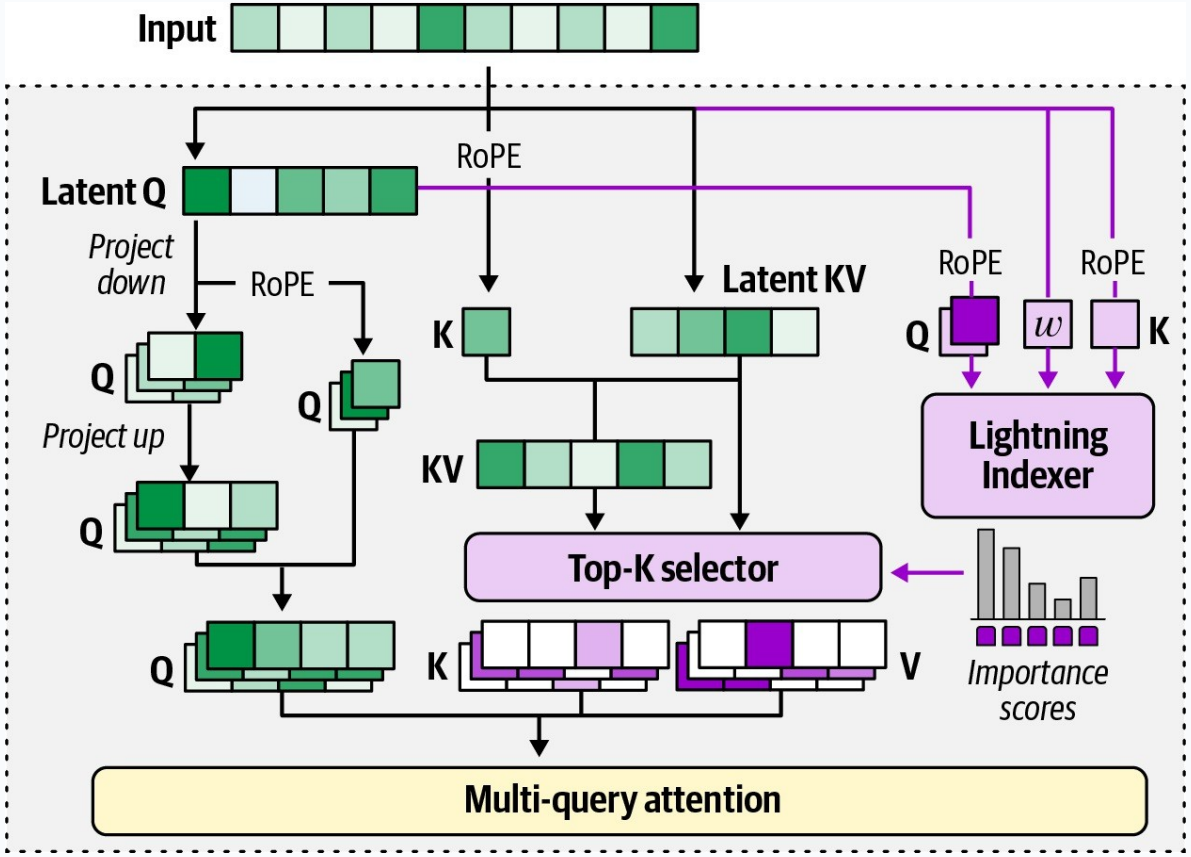
- Multi-head attention uses separate key and value representations for each attention head.
- Grouped-query and multi-query attention share key/value representations across groups or heads.
- Latent attention compresses cached information, targeting lower memory use while preserving useful contextual processing.



Original textbook illustration · PDF p. 105

Sparse Attention: Selecting Relevant Context

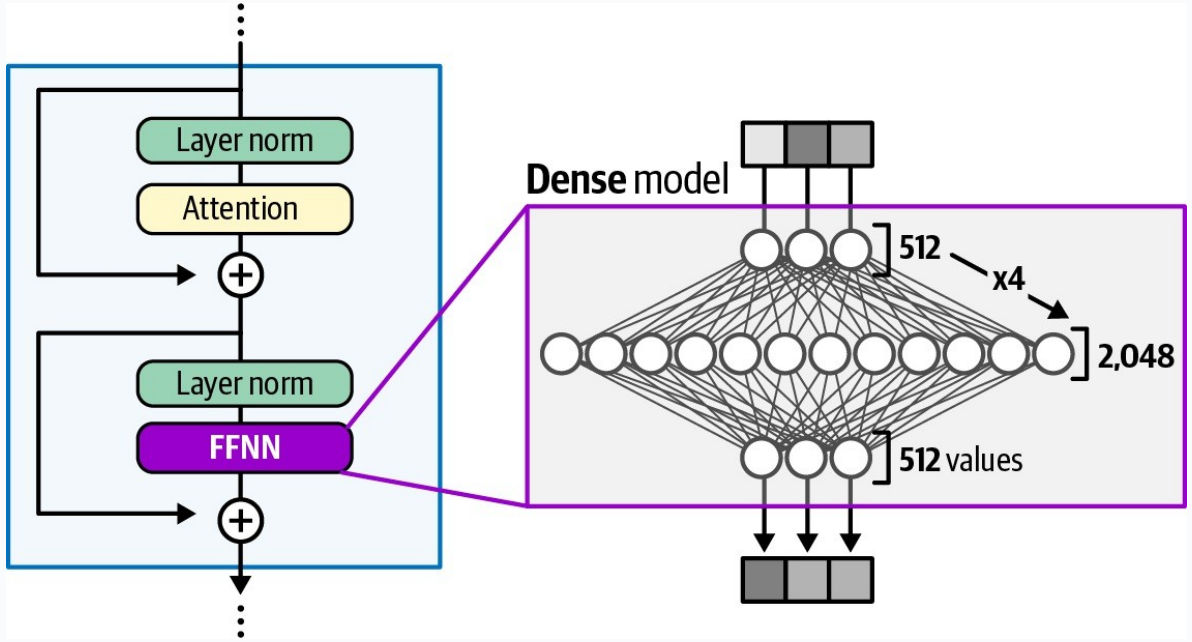
- Dense attention can require comparisons against many previous tokens.
- Sparse approaches choose a smaller subset of relevant positions for more focused computation.
- The benefit depends on how accurately the selection mechanism identifies useful context.



Original textbook illustration · PDF p. 107

Dense Versus Sparse Expert Computation

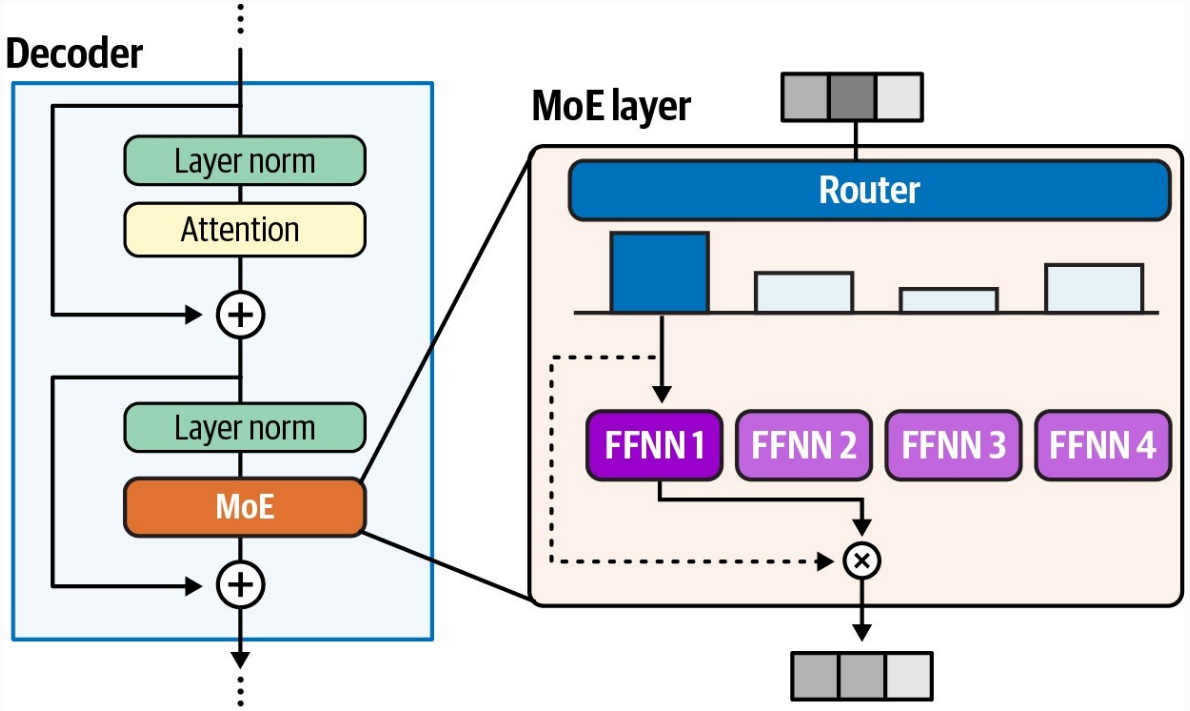
- In a dense Transformer feed-forward block, every token passes through the same network.
- In an MoE block, a router chooses which expert networks will process each token.
- Only a subset of the model's total expert parameters is active for a given token.



Original textbook illustration · PDF p. 109

The Mixture-of-Experts Router

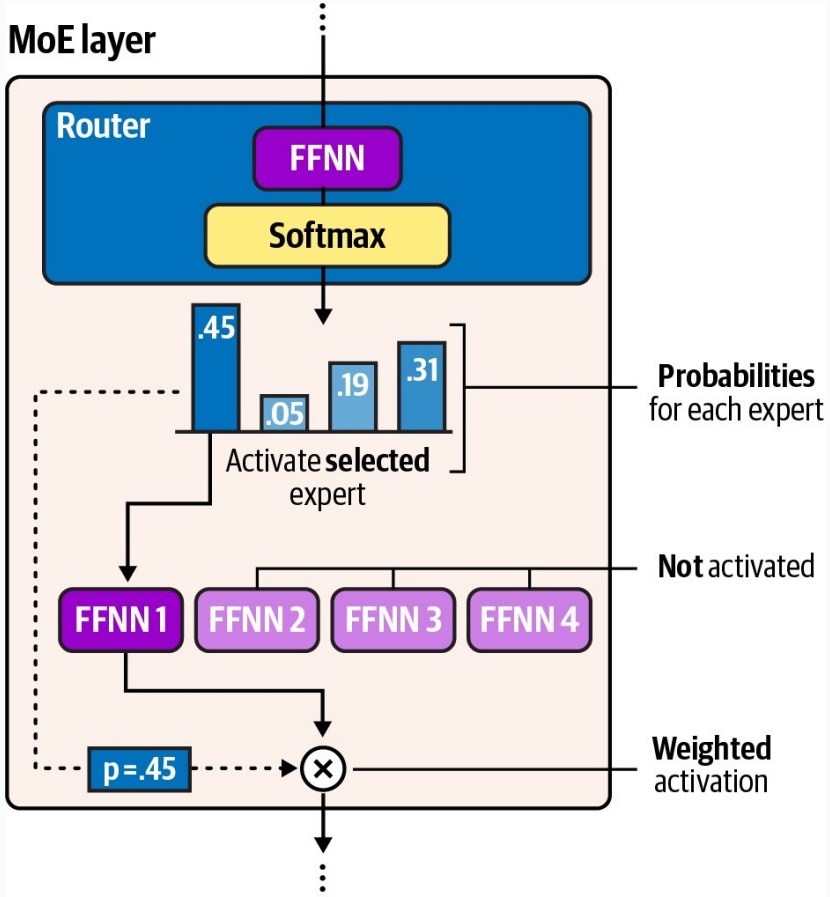
- The router computes expert scores from a token representation.
- It selects a small active set of experts and sends the token to those networks.
- The experts' outputs are combined to produce the feed-forward layer's result.



Original textbook illustration · PDF p. 111

Routing Probabilities

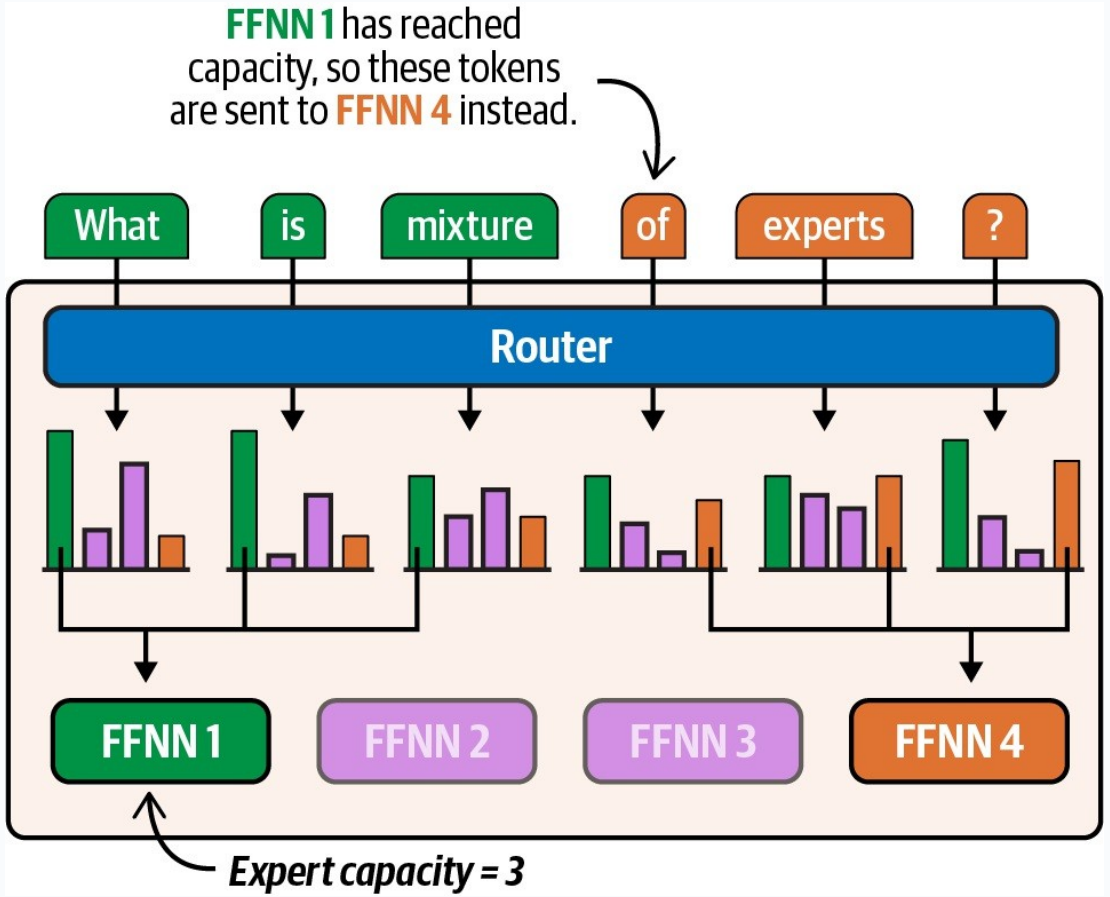
- A routing network maps token representations to scores or probabilities over experts.
- Different tokens can be sent to different experts based on their learned representations.
- Training must ensure useful specialization without allowing a few experts to absorb all traffic.



Original textbook illustration · PDF p. 112

Expert Capacity and Load Balancing

- Experts may have capacity limits that constrain how many tokens they process in a batch.
- When an expert is overloaded, routing may redirect or otherwise handle excess tokens.
- Capacity management and load balancing affect throughput, utilization, and quality.



Original textbook illustration · PDF p. 114

Total Parameters Versus Active Parameters

- An MoE model can have many total parameters because all experts contribute to its capacity.
- Only the selected experts are active for a token, so active computation is smaller than total capacity might suggest.
- The full expert set may still need to reside in memory, creating a different resource constraint.

A Large-Scale MoE Example

- Large mixture-of-experts systems can contain hundreds of experts while selecting only a small number for each token.
- This design illustrates the distinction between parameter capacity and per-token compute.
- Model selection must consider both active compute and the memory needed to host the experts.

Putting LLM Internals Into Agent Design

- Tokenization and context length shape how much evidence an agent can supply to the model.
- Decoding affects response variability; training influences instruction-following and problem-solving behavior.
- Attention, caching, and expert routing influence latency, memory consumption, and deployment choices.